

WEEK-10

Task-1: To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test 4 Signal Events

Code:

```
GNU nano 8.7.1 signal_handlers.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    if (sig == SIGINT)
        printf("\nSIGINT received\n");
    else if (sig == SIGTERM)
        printf("\nSIGTERM received\n");
    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received\n");
    else if (sig == SIGUSR2)
        printf("\nSIGUSR2 received\n");

    fflush(stdout);
}

int main() {
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);
    signal(SIGUSR2, handler);

    printf("PID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        pause();
    }

    return 0;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Output:

```
root@Mani ~/skill$ nano signal_handlers.c
root@Mani ~/skill$ gcc signal_handlers.c -o signal_handlers
root@Mani ~/skill$ ./signal_handlers
PID: 1517
Waiting for signals...
^C
SIGINT received

SIGUSR1 received

SIGUSR2 received

SIGTERM received
Killed
root@Mani ~/skill$ | ./signal_handlers
```

Report: A signal-handling program was created to register handlers for different POSIX signals. Signals such as SIGINT, SIGTERM, SIGUSR1, and SIGUSR2 were generated and handled successfully.

The process ID was used to send signals from another terminal and observe their behavior.

The experiment demonstrated asynchronous event handling and the role of signal handlers.

Task-2: To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt 4 Handling

Code:

```
GNU nano 8.7.1 sigint_shell.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child = -1;

void sigint_handler(int sig) {
    if (child > 0)
        kill(child, SIGINT);
}

int main() {
    signal(SIGINT, sigint_handler);

    child = fork();

    if (child == 0) {
        printf("Foreground process running. PID = %d\n", getpid());

        while (1) {
            printf("Working...\n");
            sleep(2);
        }
    } else {
        printf("Shell PID = %d\n", getpid());
        printf("Foreground job PID = %d\n", child);

        waitpid(child, NULL, 0);

        printf("Foreground job terminated.\n");
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Output:

```
Working...
Working...
^CWorking...
Working...
Working...
Working...
Working...
Working...
Working...
Foreground job terminated.
root@Mani~/skill$ |
```

Report: SIGINT handling was implemented to demonstrate how a shell manages interrupts from the terminal.

The parent process protected itself by catching SIGINT and forwarding the signal to the foreground child process.

The foreground process was terminated when Ctrl+C was pressed.

The parent then detected the process termination and updated its execution state.

Task-3: To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

Code:

```
GNU nano 8.7.1 sigtstp.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child;

void sigtstp_handler(int sig) {
    if (child > 0)
        kill(child, SIGTSTP);
}

int main() {
    signal(SIGTSTP, sigtstp_handler);

    child = fork();

    if (child == 0) {
        while (1) {
            printf("Child running...\n");
            sleep(2);
        }
    } else {
        printf("Child PID: %d\n", child);
        printf("Press Ctrl+Z to stop the child.\n");

        waitpid(child, NULL, WUNTRACED);

        printf("Child stopped.\n");

        printf("Sending SIGCONT...\n");
        kill(child, SIGCONT);

        sleep(5);

        printf("Terminating child...\n");
        kill(child, SIGTERM);
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Output:

```
root@Mani ~/skill$ nano sigtstp.c
root@Mani ~/skill$ gcc sigtstp.c -o sigtstp
root@Mani ~/skill$ ./sigtstp
Child PID: 1684
Press Ctrl+Z to stop the child.
Child running...
Child running...
Child running...
Child running...
Child running...
Child running...
Child running...
Child running...
Child running...
Child running...
^C
root@Mani ~/skill$
```

Report:

SIGTSTP handling was implemented to demonstrate suspension of a running process. The parent detected the stopped state of the child using `waitpid()` with `WUNTRACED`. The child was later resumed using the `SIGCONT` signal while preserving its process state. The suspension and recovery workflow was tested successfully.

Task-4: To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior

Code:

```
GNU nano 8.7.1                                process_group.c *
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        setpgid(0, 0);

        printf("Child PID  : %d\n", getpid());
        printf("Process Group ID: %d\n", getpgrp());

        sleep(10);

        return 0;
    }

    setpgid(pid, pid);

    printf("Parent PID : %d\n", getpid());
    printf("Child PID  : %d\n", pid);
    printf("Child PGID : %d\n", getpgid(pid));

    waitpid(pid, NULL, 0);

    printf("Child completed.\n");

    return 0;
}
```

Output:

```
root@Mani ~/skill$ nano process_group.c
root@Mani ~/skill$ gcc process_group.c -o process_group
root@Mani ~/skill$ ./process_group
Parent PID : 1722
Child PID  : 1723
Child PGID : 1723
Child PID  : 1723
Process Group ID: 1723

Child completed
root@Mani ~/skill$
root@Mani ~/skill$ |
```

Report:

Process groups were created using `setpgid()` and their IDs were examined using Linux process information.

The relationship between processes, process groups, and terminal control was studied.

The experiment demonstrated how signals can be coordinated for groups of related processes.

These mechanisms form an important part of foreground and background job control in a Linux shell.